



ARQUITECTURA Y SISTEMAS OPERATIVOS

Trabajo Práctico Integrador

Seguridad en Sistemas Operativos

**Alumno: Matías Fiorentini
Comisión N° 13**

Índice

1. Introducción
2. Marco Teórico
3. Caso Práctico
4. Metodología Utilizada
5. Resultados Obtenidos
6. Conclusiones
7. Bibliografía

1. Introducción

El presente trabajo integrador aborda la temática de la auditoría básica de seguridad en sistemas operativos Linux, utilizando como herramienta principal un script desarrollado en lenguaje Bash. Se eligió este tema debido a su estrecha relación con los contenidos estudiados a lo largo de la materia Arquitectura y Sistemas Operativos, en particular los módulos referidos a permisos de archivos, gestión de usuarios y análisis de registros del sistema.

La importancia de este trabajo radica en que permite aplicar de forma práctica y concreta los conceptos teóricos vistos en clase, demostrando cómo pequeñas verificaciones automatizadas pueden fortalecer la seguridad de un sistema. Conocer y poder implementar estas herramientas representa una competencia valiosa tanto en el desarrollo de software como en la administración de entornos de producción.

Los objetivos propuestos para este trabajo son los siguientes:

- Comprender los conceptos fundamentales de seguridad en sistemas operativos Linux.
- Desarrollar un script Bash que automatice una auditoría básica de seguridad.
- Aplicar los conceptos de permisos de archivos, gestión de usuarios y análisis de logs vistos en clase.
- Generar un reporte de resultados que permita identificar posibles vulnerabilidades en el sistema.

2. Marco Teórico

2.1 Seguridad en Sistemas Operativos

La seguridad en los sistemas operativos es un pilar fundamental para proteger la información y garantizar la estabilidad de los sistemas. Según los materiales de la cátedra, cada dispositivo conectado a una red representa una puerta de entrada potencial que debe ser protegida mediante estrategias preventivas y reactivas.

Un sistema operativo seguro debe ser capaz de proteger la información de accesos no autorizados, garantizar que los recursos del sistema no sean modificados sin permiso, y mantenerse estable ante posibles ataques.

2.2 Permisos de Archivos en Linux

Linux implementa un modelo de permisos basado en tres categorías de usuarios: el propietario del archivo (user), el grupo al que pertenece (group), y el resto de usuarios del sistema (others). Para cada categoría se pueden asignar tres tipos de permisos:

- Lectura (r): permite ver el contenido de un archivo o listar un directorio.
- Escritura (w): permite modificar el contenido de un archivo.
- Ejecución (x): permite ejecutar un archivo como programa o ingresar a un directorio.

Estos permisos se expresan en notación octal, es decir con números del 0 al 7. Cada número representa una combinación de esos tres permisos. El 4 significa solo lectura, el 2 significa solo escritura, el 1 significa solo ejecución. Y cuando se combinan se suman. Por ejemplo el 6 es 4 más 2, lo que significa lectura y escritura. El 7 es 4 más 2 más 1, lo que significa lectura, escritura y ejecución. Y el 0 significa ningún permiso.

Entonces cuando vemos un permiso de tres dígitos como 644, cada dígito representa un tipo de usuario. El primer dígito es para el propietario, el segundo para el grupo y el tercero para el resto del sistema. En el caso de 644: el propietario tiene un 6, que es lectura y escritura. El grupo tiene un 4, que es solo lectura. Y el resto también tiene un 4, solo lectura. Esto significa que solo el dueño del archivo puede modificarlo, todos los demás solo pueden leerlo.

En cambio, cuando un archivo tiene permisos 777, los tres dígitos son 7, lo que significa que cualquier usuario del sistema puede leer, escribir y ejecutar ese archivo.

Los archivos más sensibles del sistema son **/etc/passwd**, que contiene el listado de usuarios, y **/etc/shadow**, que almacena las contraseñas cifradas. Sus permisos correctos son 644 y 640 respectivamente.

2.3 Gestión de Usuarios y Contraseñas

En Linux cada usuario posee un identificador único llamado UID. Existen dos tipos de usuarios bien diferenciados:

- Usuarios humanos: se autentican con contraseña y utilizan el sistema de forma interactiva.
- Usuarios de servicio: el sistema los crea automáticamente durante la instalación para que ciertos procesos internos funcionen de forma aislada y con permisos limitados. Nunca inician sesión de forma manual.

- Fuentes: <https://itsfoss.com/es/uid-en-linux/> - <https://linuxize.com/post/how-to-create-users-in-linux-using-the-useradd-command/>

¿Cómo puede un usuario quedarse sin contraseña?

Existen varias situaciones en las que un usuario del sistema puede no tener contraseña asignada:

- El sistema lo crea así por diseño: los usuarios de servicio como syslog, avahi o gdm se crean sin contraseña desde el principio porque nunca necesitan autenticarse de forma interactiva.
- Un administrador crea un usuario y olvida asignarle contraseña: al ejecutar **sudo useradd juan** el usuario queda creado pero sin contraseña hasta que se ejecute **sudo passwd juan**.
- Un administrador elimina la contraseña intencionalmente: el comando **sudo passwd -d juan** borra la contraseña de un usuario existente. Esto suele hacerse en entornos de prueba y a veces se olvida revertir en producción.

El riesgo real depende del tipo de cuenta: un usuario de servicio sin contraseña es lo esperado y no representa un problema. En cambio, un usuario humano sin contraseña es una vulnerabilidad crítica, ya que cualquier persona podría acceder al sistema simplemente ingresando el nombre de usuario y dejando la contraseña en blanco.

2.4 Análisis de Logs y Auditoría

Los logs o registros del sistema son archivos que almacenan todos los eventos ocurridos en el sistema operativo. Su análisis es fundamental para detectar actividades sospechosas e intentos de acceso no autorizado. En sistemas Ubuntu, el archivo **/var/log/auth.log** registra todos los eventos relacionados con la autenticación: inicios de sesión exitosos, intentos fallidos, uso de sudo, entre otros.

El análisis periódico de estos registros permite al administrador del sistema tomar decisiones informadas sobre el estado de seguridad, actuando de forma preventiva ante cualquier actividad inusual.

2.5 Automatización con cron

Cron es el planificador de tareas de Linux. Permite programar la ejecución automática de comandos o scripts en intervalos regulares, sin necesidad de intervención manual. Cada tarea programada se define mediante una línea en el archivo **crontab**, con el siguiente formato:

```
minuto hora día mes día_semana comando
```

Por ejemplo, para ejecutar un script todos los días a las 3 de la mañana:

```
0 3 * * * /usr/local/sbin/auditoria_seguridad.sh
```

Esta herramienta es fundamental en la administración de servidores porque permite automatizar tareas de mantenimiento y seguridad que de otro modo dependerían de la memoria del administrador.

2.6 Principio de Mínimo Privilegio

El principio de mínimo privilegio establece que cada usuario o proceso debe contar únicamente con los permisos estrictamente necesarios para realizar su función. Este principio reduce la superficie de ataque: si una cuenta es comprometida, el atacante solo tendrá acceso a los recursos que esa cuenta tenía permitidos. En la práctica se aplica mediante la asignación de permisos con `chmod` y la revisión periódica de configuraciones del sistema.

3. Caso Práctico

3.1 Descripción del Problema

El caso que se planteó fue el siguiente: una empresa cuenta con un servidor Linux al que acceden múltiples usuarios. El administrador del sistema, también llamado `sysadmin`, es el responsable de garantizar que ese servidor esté correctamente configurado y protegido. Para cumplir esa tarea necesita verificar periódicamente tres aspectos clave: si existen usuarios sin contraseña, si los archivos críticos tienen los permisos correctos, y si hubo intentos de acceso no autorizado.

En lugar de realizar estas verificaciones manualmente, se desarrolló un script Bash llamado **`auditoria_seguridad.sh`** que las automatiza y genera un reporte con los resultados. El script es ejecutado por el administrador con permisos de superusuario, ya que necesita acceder a archivos del sistema que no están disponibles para usuarios comunes.

Para que la auditoría sea continua y no dependa de que el administrador lo recuerde, el script se programa usando `cron`, el planificador de tareas de Linux. Esto permite configurarlo para que se ejecute automáticamente todos los días a las 3 de la mañana, cuando el servidor tiene menos actividad:

```
0 3 * * * /usr/local/sbin/auditoria_seguridad.sh
```

Cada mañana el administrador revisa el reporte generado durante la noche. En organizaciones más grandes, ese reporte puede enviarse automáticamente por correo electrónico al equipo de seguridad, garantizando que cualquier anomalía sea detectada y atendida a tiempo.

Para simular un entorno de Linux se usó **DistroSea** (<https://distrosea.com>), una plataforma online que provee acceso a sistemas operativos reales a través del navegador sin necesidad de instalación local. Se utilizó una instancia de Ubuntu 25 con terminal Bash.

Las tres verificaciones implementadas son:

- Verificación de usuarios sin contraseña: detecta cuentas que podrían permitir acceso sin autenticación.
- Revisión de permisos de archivos críticos: comprueba que `/etc/passwd` y `/etc/shadow` tengan los permisos correctos.
- Análisis de intentos de login fallidos: extrae del log de autenticación los últimos intentos fallidos registrados.

3.2 Script desarrollado: auditoria_seguridad.sh

A continuación, se muestra el script completo desarrollado para este trabajo:

```
GNU nano 8.3 auditoria_seguridad.sh
#!/bin/bash
#Script de auditoria básica de seguridad

REPORTE="reporte_auditoria.txt"
FECHA=$(date "+%d/%m/%Y %H:%M:%S")

echo "===== " > $REPORTE
echo " REPORTE DE AUDITORÍA DE SEGURIDAD" >> $REPORTE
echo " Fecha: $FECHA" >> $REPORTE
echo "===== " >> $REPORTE
echo "" >> $REPORTE

# --- BLOQUE 1: Usuarios sin contraseña ---
echo "[1] USUARIOS SIN CONTRASEÑA:" >> $REPORTE
usuarios_sin_pass=$(awk -F: '($2 == "" || $2 == "!") {print $1}' /etc/shadow 2>/dev/null)
if [ -z "$usuarios_sin_pass" ]; then
    echo "OK - No se encontraron usuarios sin contraseña." >> $REPORTE
else
    echo "ALERTA - Usuarios sin contraseña: $usuarios_sin_pass" >> $REPORTE
fi
echo "" >> $REPORTE

# ---BLOQUE 2: Permisos de archivos criticos ---
echo "[2] PERMISOS DE ARCHIVOS CRÍTICOS:" >> $REPORTE
perms_passwd=$(stat -c "%a" /etc/passwd)
if [ "$perms_passwd" = "644" ]; then
    echo "OK - /etc/passwd tiene permisos correctos (644)" >> $REPORTE
else
    echo "ALERTA - /etc/passwd tiene permisos: $perms_passwd (esperando: 644)" >> $REPORTE
fi

perms_shadow=$(stat -c "%a" /etc/shadow)
if [ "$perms_shadow" = "640" ] || [ "$perms_shadow" = "000" ]; then
    echo "OK - /etc/shadow tiene permisos correctos ($perms_shadow)" >> $REPORTE
else
    echo "ALERTA - /etc/shadow tiene permisos: $perms_shadow (esperando: 640)" >> $REPORTE
fi
echo "" >> $REPORTE

# --- BLOQUE 3: Intentos de login fallidos ---
echo "[3] ÚLTIMOS INTENTOS DE LOGIN FALLIDOS:" >> $REPORTE
if [ -f /var/log/auth.log ]; then
    intentos=$(grep "authentication failure" /var/log/auth.log | tail -5)
    if [ -z "$intentos" ]; then
        echo "OK - No se encontraron intentos fallidos recientes." >> $REPORTE
    else
        echo "$intentos" >> $REPORTE
    fi
else
    echo "INFO - Archivos auth.log no encontrado en este entorno" >> $REPORTE
fi
echo "" >> $REPORTE

echo "===== " >> $REPORTE
echo " Auditoría finalizada." >> $REPORTE
echo "===== " >> $REPORTE

echo ""
echo "Auditoría completada. Ver reporte: $REPORTE"
```

3.3 Explicación de los comandos utilizados

nano

Editor de texto en terminal. Se utilizó para crear y editar el archivo del script directamente desde la línea de comandos, sin necesidad de interfaz gráfica ni IDE externo.

```
ubuntu@ubuntu:~$ nano auditoria_seguridad.sh
```

chmod

Comando que modifica los permisos de un archivo. Se usó de dos formas: para dar permisos de ejecución al script, y para simular una configuración insegura en las pruebas.

```
ubuntu@ubuntu:~$ chmod +x auditoria_seguridad.sh
```

```
ubuntu@ubuntu:~$ sudo chmod 777 /etc/passwd
```

```
ubuntu@ubuntu:~$ sudo chmod 644 /etc/passwd
```

echo

Imprime texto en pantalla o lo redirige a un archivo. En el script se usa con dos operadores: el símbolo > crea el archivo y escribe en él, mientras que >> agrega contenido al final sin borrar lo existente.

```
echo "===== " > $REPORTE
echo " REPORTE DE AUDITORÍA DE SEGURIDAD" >> $REPORTE
echo " Fecha: $FECHA" >> $REPORTE
echo "===== " >> $REPORTE
echo "" >> $REPORTE
```

date

Obtiene la fecha y hora actual del sistema. En el script se usa para registrar cuándo se realizó la auditoría.

```
FECHA=$(date "+%d/%m/%Y %H:%M:%S")
```

awk

Herramienta para procesar texto línea por línea. En el script se usa para leer /etc/shadow, separar sus columnas por el carácter : y detectar si la segunda columna está vacía o contiene !, lo que indica que el usuario no tiene contraseña asignada.

```
usuarios_sin_pass=$(awk -F: '($2 == "" || $2 == "!") {print $1}' /etc/shadow 2>/dev/null)
```


stat

Muestra información detallada de un archivo. Con la opción -c "%a" devuelve los permisos en formato octal, lo que permite comparar si el valor es el esperado.

```
perms_passwd=$(stat -c "%a" /etc/passwd)
```

grep

Busca líneas que contengan un texto específico dentro de un archivo. En el script se usa para filtrar las líneas del log que contienen el texto authentication failure.

```
intentos=$(grep "authentication failure" /var/log/auth.log | tail -5)
```

tail

Muestra las últimas líneas de un archivo o de la salida de otro comando. Se combina con grep mediante el operador | (pipe) para mostrar solo los últimos 5 intentos fallidos.

```
intentos=$(grep "authentication failure" /var/log/auth.log | tail -5)
```

su

Permite cambiar de usuario en la terminal. Se utilizó para simular intentos de autenticación fallidos ingresando contraseñas incorrectas, lo que generó registros en el log del sistema.

```
ubuntu@ubuntu:~$ su root
Password:
su: Authentication failure
```

cat

Muestra el contenido completo de un archivo en pantalla. Se usó para visualizar el reporte generado luego de cada ejecución.

```
ubuntu@ubuntu:~$ cat reporte_auditoria.txt
```

3.4 Instrucciones de Ejecución

Para ejecutar el script en el entorno Ubuntu de DistroSea se siguieron los siguientes pasos:

1. Crear el archivo con el editor nano: **nano auditoria_seguridad.sh**
2. Desarrollo del script bash -> código.
3. Dar permisos de ejecución: **chmod +x auditoria_seguridad.sh**
4. Ejecutar el script con permisos de superusuario: **sudo .auditoria_seguridad.sh**
5. Visualizar el reporte generado: **cat reporte_auditoria.txt**

4. Metodología Utilizada

4.1 Investigación Previa

Para el desarrollo del trabajo se consultaron los materiales proporcionados por la cátedra, en particular los módulos sobre introducción a la seguridad en sistemas operativos, principales vulnerabilidades, herramientas de seguridad y seguridad avanzada, como también a los documentos sobre línea de comandos. Además se consultaron fuentes externas, agregadas en la sección bibliografía

4.2 Herramientas Utilizadas

- Sistema operativo: Ubuntu 25 provisto por DistroSea
- Lenguaje de scripting: Bash.
- Editor de texto: nano, editor de terminal integrado en Ubuntu.
- Comandos principales: nano, chmod, echo, date, awk, stat, grep, tail, su y cat.

4.3 Etapas de Desarrollo

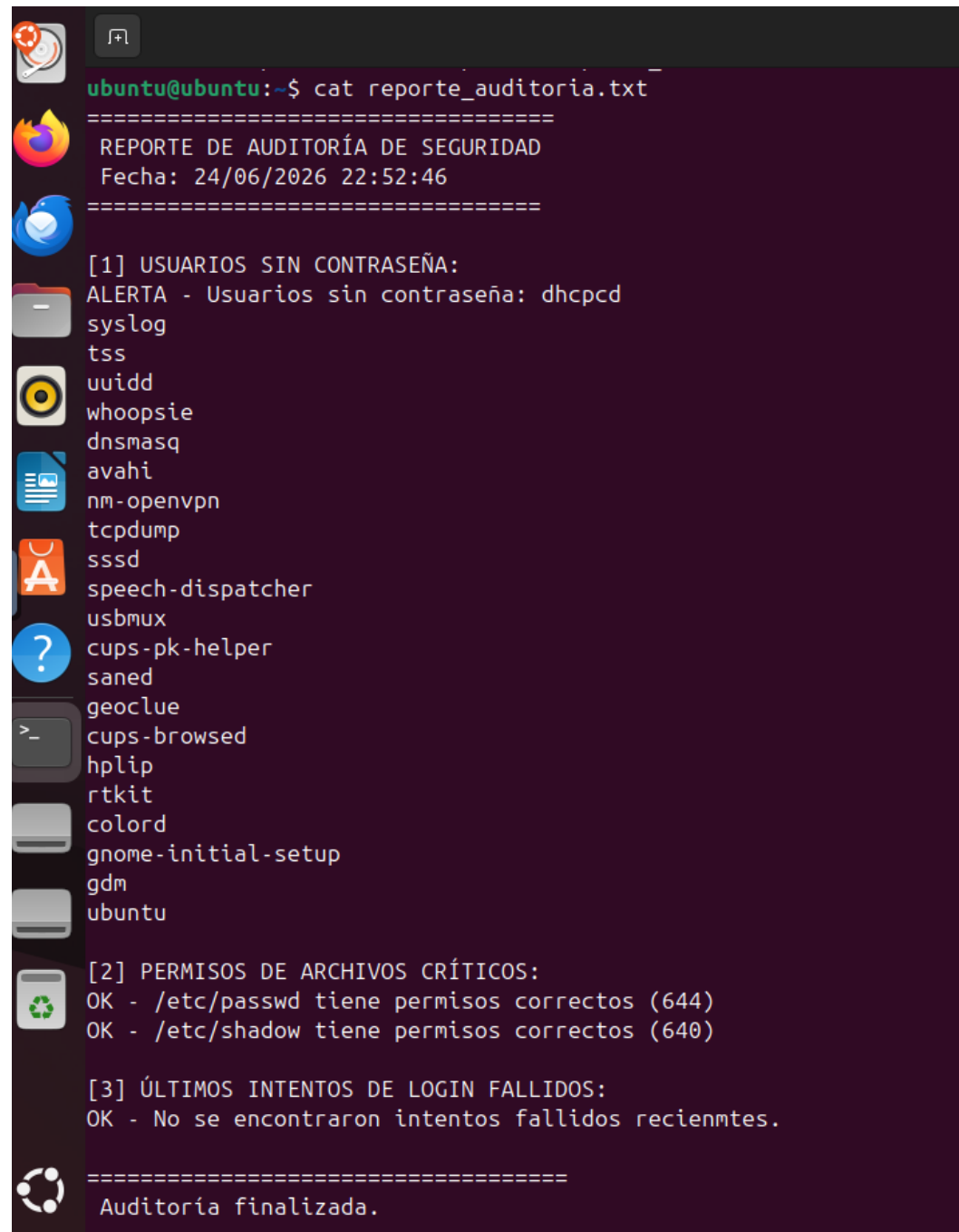
El trabajo se desarrolló en las siguientes etapas:

- **Planificación:** se definieron las tres verificaciones a implementar en base a los conceptos teóricos estudiados.
- **Desarrollo del script:** se escribió cada bloque usando el editor nano en la terminal de DistroSea.
- **Prueba normal:** se ejecutó el script para verificar el comportamiento con el sistema en estado correcto.
- **Pruebas de alerta:** se simulaban situaciones inseguras para validar que el script detectaba y reportaba correctamente los problemas.
- **Documentación:** se redactó el presente informe integrando los resultados y capturas obtenidas.

5. Resultados Obtenidos

5.1 Ejecución normal del script

Al ejecutar el script por primera vez sobre el sistema Ubuntu de DistroSea sin ninguna modificación previa, se obtuvo el siguiente resultado:



```
ubuntu@ubuntu:~$ cat reporte_auditoria.txt
=====
REPORTE DE AUDITORÍA DE SEGURIDAD
Fecha: 24/06/2026 22:52:46
=====

[1] USUARIOS SIN CONTRASEÑA:
ALERTA - Usuarios sin contraseña: dhcpcd
syslog
tss
uidd
whoopsie
dnsmasq
avahi
nm-openvpn
tcpdump
sssd
speech-dispatcher
usbmux
cups-pk-helper
saned
geoclue
cups-browsed
hplip
rtkit
colord
gnome-initial-setup
gdm
ubuntu

[2] PERMISOS DE ARCHIVOS CRÍTICOS:
OK - /etc/passwd tiene permisos correctos (644)
OK - /etc/shadow tiene permisos correctos (640)

[3] ÚLTIMOS INTENTOS DE LOGIN FALLIDOS:
OK - No se encontraron intentos fallidos recientes.

=====
Auditoría finalizada.
=====
```

El resultado mostró una alerta en el Bloque 1 por usuarios de servicio del sistema, permisos correctos en el Bloque 2 y sin intentos fallidos en el Bloque 3.

5.2 Bloque 1 – Usuarios sin contraseña

El Bloque 1 detectó una lista de usuarios sin contraseña asignada, entre ellos: dhcpcd, syslog, tss, uidd, whoopsie, avahi, gdm y ubuntu, entre otros. Sin embargo, esto no representa una vulnerabilidad real en el sistema.

Todos los usuarios listados son cuentas de servicio que el propio sistema operativo Ubuntu crea automáticamente durante la instalación. Estas cuentas no están diseñadas para que una persona inicie sesión con ellas, sino para que ciertos procesos internos del sistema funcionen de forma aislada y con permisos limitados. Por ese motivo no tienen contraseña asignada.

Esta situación demuestra una limitación del script: al verificar **/etc/shadow** no diferencia entre usuarios humanos y usuarios de servicio. En un entorno real, un administrador debería analizar la lista y determinar cuáles representan un riesgo.

5.3 Bloque 2 – Permisos de archivos críticos

Para validar el funcionamiento del Bloque 2 se modificó intencionalmente el permiso de **/etc/passwd** a 777, lo que representa una configuración insegura ya que cualquier usuario del sistema podría modificar ese archivo. El script detectó correctamente la situación y registró la alerta en el reporte. Luego se restauró el permiso original a 644.

```
ubuntu@ubuntu:~$ sudo chmod 777 /etc/passwd
ubuntu@ubuntu:~$ sudo ./auditoria_seguridad.sh

Auditoría completada. Ver reporte: reporte_auditoria.txt

ubuntu@ubuntu:~$ cat reporte_auditoria.txt
=====
REPORTE DE AUDITORÍA DE SEGURIDAD
Fecha: 24/06/2026 22:57:46
=====

[1] USUARIOS SIN CONTRASEÑA:
ALERTA - Usuarios sin contraseña: dhcpcd
syslog
tss
uidd
whoopsie
dnsmasq
avahi
nm-openvpn
tcpdump
sssd
speech-dispatcher
usbmux
cups-pk-helper
saned
geoclue
cups-browsed
hplip
rtkit
colord
gnome-initial-setup
gdm
ubuntu

[2] PERMISOS DE ARCHIVOS CRÍTICOS:
ALERTA - /etc/passwd tiene permisos: 777 (esperando: 644)
OK - /etc/shadow tiene permisos correctos (640)

[3] ÚLTIMOS INTENTOS DE LOGIN FALLIDOS:
OK - No se encontraron intentos fallidos recientes.

=====
Auditoría finalizada.
=====
```

5.4 Bloque 3 – Intentos de login fallidos

Para generar intentos fallidos de autenticación en el log del sistema se utilizó el comando `su root` ingresando contraseñas incorrectas de forma repetida. Esto generó registros del tipo **authentication failure** en el archivo `/var/log/auth.log`.

```
ubuntu@ubuntu:~$ cat reporte_auditoria.txt
```

```
=====
REPORTE DE AUDITORÍA DE SEGURIDAD
Fecha: 24/06/2026 23:09:19
=====
```

[1] USUARIOS SIN CONTRASEÑA:

ALERTA - Usuarios sin contraseña: dhcpd

syslog

tss

uidd

whoopsie

dnsmasq

avahi

nm-openvpn

tcpdump

sssd

speech-dispatcher

usbmux

cups-pk-helper

saned

geoclue

cups-browsed

hplip

rtkit

colord

gnome-initial-setup

gdm

ubuntu

[2] PERMISOS DE ARCHIVOS CRÍTICOS:

OK - /etc/passwd tiene permisos correctos (644)

OK - /etc/shadow tiene permisos correctos (640)

[3] ÚLTIMOS INTENTOS DE LOGIN FALLIDOS:

```
2026-06-24T23:06:41.218942+00:00 ubuntu su: pam_unix(su:auth): authentication failure; logname=ubuntu uid=1000 euid=0 tty=/dev/pts/0 ruser=ubuntu rhost= user=root
2026-06-24T23:06:48.275884+00:00 ubuntu su: pam_unix(su:auth): authentication failure; logname=ubuntu uid=1000 euid=0 tty=/dev/pts/0 ruser=ubuntu rhost= user=root
2026-06-24T23:06:55.478181+00:00 ubuntu su: pam_unix(su:auth): authentication failure; logname=ubuntu uid=1000 euid=0 tty=/dev/pts/0 ruser=ubuntu rhost= user=root
2026-06-24T23:07:01.881661+00:00 ubuntu su: pam_unix(su:auth): authentication failure; logname=ubuntu uid=1000 euid=0 tty=/dev/pts/0 ruser=ubuntu rhost= user=root
2026-06-24T23:07:58.104649+00:00 ubuntu sudo: ubuntu : TTY=pts/0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/grep 'authentication failure' /var/log/auth.log
```

```
=====
Auditoria finalizada.
=====
```

5.5 Dificultades Encontradas

Durante el desarrollo se presentaron las siguientes dificultades que pudieron resolverse:

- El script originalmente buscaba la cadena 'Failed password' en el log, pero en este entorno Ubuntu los intentos fallidos se registran como 'authentication failure'. Se actualizó el script para adaptarlo al formato correcto.
- El archivo /etc/shadow solo es legible por root, por lo que fue necesario ejecutar el script con sudo.

6. Conclusiones

La realización de este trabajo me permitió integrar los conceptos teóricos estudiados durante la materia a través de una aplicación práctica. Con el desarrollo del script **auditoria_seguridad.sh** pude comprobar cómo herramientas simples del sistema operativo, combinadas con scripting Bash, son capaces de automatizar tareas de verificación de seguridad que de otro modo deberían realizarse manualmente.

Uno de los aprendizajes más relevantes fue comprender la diferencia entre usuarios humanos y usuarios de servicio en Linux, y cómo una herramienta automatizada puede generar falsos positivos si no se tiene ese contexto. Esto demuestra que las herramientas de auditoría son un punto de partida, pero siempre requieren del criterio de un administrador para interpretar correctamente los resultados.

Trabajar con permisos de archivos me permitió poner en práctica uno de los conceptos más importantes vistos en la materia. Configurar, verificar y corregir permisos sobre archivos reales del sistema me permitió comprender por qué este concepto es tan importante dentro de la seguridad en sistemas operativos.

También, el análisis de logs me resultó valioso como aprendizaje. Entendí que el sistema operativo registra constantemente lo que ocurre, y que revisar esos registros es una forma concreta de saber si algo inusual está pasando.

Por último, quiero mencionar que este trabajo fue lo realicé de forma individual. Si bien la consigna contemplaba la posibilidad de trabajar en grupo, no logré conformar un equipo, por lo que todo el proceso de investigación, desarrollo del script, pruebas y documentación estuvo a mi cargo. Esto representó un desafío adicional pero también una oportunidad para profundizar de manera más completa cada uno de los conceptos trabajados, ya que no tuve la posibilidad de delegar ninguna etapa del proceso.

En conclusión, este trabajo demuestra que la seguridad en sistemas operativos no es un estado fijo sino un proceso continuo que requiere verificación, mantenimiento y mejora constante.

7. Bibliografía

- Cátedra Arquitectura y Sistemas Operativos UTN (2024). Introducción a la seguridad en sistemas operativos. Material de estudio, Tecnicatura Universitaria en Programación a Distancia.
- Cátedra Arquitectura y Sistemas Operativos UTN (2024). Seguridad en sistemas operativos. Material de estudio, Tecnicatura Universitaria en Programación a Distancia.
- Cátedra Arquitectura y Sistemas Operativos UTN (2024). Seguridad avanzada y mantenimiento preventivo. Material de estudio, Tecnicatura Universitaria en Programación a Distancia.
- <https://www.freecodecamp.org/espanol/news/comando-chmod-como-cambiar-permisos-de-archivo-en-linux/>
- https://www.linuxtotal.com.mx/index.php?cont=info_admon_011
- <https://www.fing.edu.uy/inco/cursos/sistoper/recursosLaboratorio/tutorial0.pdf>
- <https://www.freecodecamp.org/espanol/news/comandos-de-linux/>
- <https://www.welivesecurity.com/la-es/2014/06/17/10-comandos-de-linux-gestionar-seguridad/>
- <https://www.freecodecamp.org/espanol/news/tutorial-de-programacion-de-bash-script-de-shell-de-linux-y-linea-de-comandos-para-principiantes/>
- https://www.w3schools.com/bash/bash_cron.php
- DistroSea. (2024). Online Linux environment. Recuperado de <https://distrosea.com> [Acceso: junio 2025]